

HelmNet

PGP-AIML-BA-UTA-Jun25-D

01/12/2026

Yashpal Singh

Contents / Agenda

- Exeulative Summary
- Data Overview
- Exploratory Data Analysis
- Data Pre-processing
- Basic Convolutional Neural Network
- VGG-16 Model
- VGG-16 with FFNN
- VGG-16 with Data Augmentation
- Model Performance Comparison and Final Model Selection

Executive Summary: HelmNet Safety Compliance System

1. Project Overview :

SafeGuard Corp identified a critical safety gap in high-risk industrial environments where manual monitoring for safety helmet compliance is prone to human error, fatigue, and "blind spots". The **HelmNet project** was launched to develop an automated, real-time computer vision solution to ensure 100% compliance with Personal Protective Equipment (PPE) protocols, thereby mitigating the risk of workplace fatalities and long-term disabilities.

2. Business Problem and Objectives

Manual oversight often leads to a **delayed response**, identifying non-compliance only after an incident has occurred. Our technical objective was to architect a high-precision binary classifier—**With Helmet (Class 1)** vs. **Without Helmet (Class 0)**—that remains robust across diverse lighting, angles, and postural variations.

3. Data Insights & Strategy

The system was trained on a balanced dataset of **631 high-diversity images (311 With Helmet, 320 Without Helmet)**

- **Near-Perfect Class Balance:** This symmetry (51% to 49%) prevented statistical bias toward a majority class, ensuring that our performance metrics—especially Accuracy and Recall—are highly reliable.
- **Preprocessing:** We utilized **Gray Scaling** for computational efficiency and **Standardization** (scaling pixels to 0–1) for faster convergence

Executive Summary: HelmNet Safety Compliance System

4. Model Evolution & Final Selection:

A four-stage experimental roadmap is conducted:

1. **Base CNN:** Established a strong baseline but **lacked the depth for complex features**.
2. **VGG-16 (Base):** Showed signs of underfitting due to a lack of complexity in its classification head.
3. **VGG-16 + Data Augmentation:** Highly accurate on training data (**0.978**) but showed slight "**memorization**" behavior.
4. **Final Choice—VGG-16 with FFNN (Feed-Forward Neural Network):** This architecture emerged as the "**Winner**". It achieved the best **balance of safety and reliability**, capturing complex spatial relationships with **exceptional generalization**.

5. Performance Results:

The selected **VGG-16 + FFNN** model demonstrated **flawless reliability on unseen data**:

- **Validation Accuracy/Recall/F1: 0.9841.**
- **Test Results:** Achieved a **perfect score of 1.0** across all key metrics (**Accuracy, Precision, Recall, and F1-Score**) on the final test set.
- **Safety-First Reliability:** The model eliminated both False Positives and False Negatives, demonstrating it has reached **maximum discriminative capacity for this environment**

Executive Summary: HelmNet Safety Compliance System

6. Actionable Insights & Recommendations

- ✓ **Deploy VGG-16 + FFNN:** The model is ready for integration into **SafeGuard** Corp's monitoring infrastructure due to its perfect generalization and **zero "False Negative" rate during testing.**
- ✓ **Prioritize Recall for Compliance:** In future iterations, continue to weight **Recall** as the primary safety metric to ensure no un-helmeted worker goes undetected.
- ✓ **Scalability:** The model's **success with "action-oriented"** images confirms it can be scaled to monitor workers in motion, not just in static poses.
- ✓ **Continuous Improvement:** While the current model is perfect for this dataset, we recommend ongoing data collection of edge cases (e.g., extremely low light or obscured views) to maintain this high performance in evolving site conditions.

Business Problem - HelmNet

Business Context :

Workplace safety in hazardous environments like construction sites and industrial plants is crucial to prevent accidents and injuries. One of the most important safety measures is ensuring workers wear safety helmets, which protect against head injuries from falling objects and machinery. Non-compliance with helmet regulations increases the risk of serious injuries or fatalities, making effective monitoring essential, especially in large-scale operations where manual oversight is prone to errors and inefficiency.

Problem Statement :

The challenge of scalable safety compliance

In high-risk industrial and construction environments, head injuries remain a leading cause of workplace fatalities and long-term disability. While safety helmets are a mandatory defense, **manual monitoring of compliance** in large-scale operations is fundamentally flawed due to:

- **Human Oversight Limitations:** Safety officers cannot monitor every zone simultaneously, leading to "blind spots" in enforcement.
- **Operational Inefficiency:** Manual checks are labor-intensive and prone to fatigue-driven errors.
- **Delayed Response:** Non-compliance is often identified only after an incident occurs, rather than through proactive prevention.

There is a critical need for **an automated, computer-vision-based auditing solution** that provides consistent, real-time oversight to mitigate these risks and ensure a zero-harm work culture.

Business Problem - HelmNet

Project Objective :

Development of an Automated Vision System for Safety Enforcement

The primary goal of this project is to architect and train a high-precision **Deep Learning Image classification model** capable of distinguishing between **compliant and non-compliant safety practices through image analysis**.

Key Technical Milestones :

- ✓ **Robust Classification:** Develop a Binary Classifier to accurately categorize images into With Helmet and Without Helmet across diverse environmental conditions (varying lighting, angles, and distances).
- ✓ **Evaluation Excellence:** Achieve high **Recall** scores to minimize "False Negatives" (failing to detect a worker without a helmet), as these represent the highest risk to the organization supported by the **F1 Score**

Solution Approach

1. Data Strategy & Exploratory Analysis (EDA) : *Building a Foundation of Data Integrity.*

- **Quality Audit:** Analyzing dataset uniqueness to prevent data leakage and checking for class imbalance

2. Data Preprocessing :

- **Normalization:** Scaling pixel values to 0,1 for faster gradient convergence.
- **Dimensionality:** Standardizing image resolution for model input.
- **Gray Scaling Analysis:** Evaluating if color is a critical feature (e.g., high-visibility yellow/orange) or if geometric features (the shape of the helmet) are sufficient.

3. Split Strategy: Using an **80/10/10** (Train/Val/Test) split to ensure the final evaluation is performed on completely **unseen data**

4. The Modeling Evolution (Architectural Roadmap) : *From Baseline to transfer learning*

Four-stage experimental design to identify the most robust architecture

- **Baseline (Simple CNN):** A custom-built shallow network to establish a performance floor.
- **Transfer Learning (VGG-16 Base):** Leveraging spatial hierarchies learned from ~1.4 million images (*ImageNet*).
- **Enhanced VGG-16 (+FFNN):** Replacing the standard head with a custom **Feed-Forward Neural Network** to capture complex relationships in safety gear features.
- **Regularized VGG-16 (+Data Augmentation):** Stress-testing the model by simulating environmental variances (rotations, zoom, and shifts) to ensure **Translational Invariance**.

Solution Approach

5. Results & Decision Logic: Validation Performance & Model Selection.

- **Evaluation Excellence:** Compare above trained model and achieve high ,Accuracy(general reliability) ,**Recall** scores to minimize "**False Negatives**" (**failing to detect a worker without a helmet**), as these represent the highest risk to the organization followed by the **F1 Score** and get finalized the best model

Why these metrics matter most and aligned with project objective

1. The Priority : Recall(Sensitivity)

In a safety context, **Recall** is the most critical metric. Recall answers the question: "*Of all the workers actually not wearing a helmet, how many did the system correctly flag?*"

- **The Goal:** Minimize **False Negatives** (workers without helmets who are marked as "safe" or missed by the system).
- **The Stakes:** A False Negative means a **worker enters a hazardous zone unprotected**, which could lead to a fatality. This is a catastrophic failure for SafeGuard Corp.
- **The Trade-off:** You would rather have the **system "over-alert"** (flagging someone with a bulky hat as a violation) than "**under-alert**" (missing a person with no protection).

2. The Balancer : F1 Score

While Recall prevents accidents, **Precision** can not be ignored entirely. If model has 100% Recall but 5% Precision, it will trigger false alarms every 30 seconds, leading to "**alarm fatigue**" where supervisors eventually ignore the system.

- **The Role of F1:** The F1-Score is the **harmonic mean** of Precision and Recall.
- **Why it's better than Accuracy:** Accuracy is misleading if dataset is imbalanced (e.g., if 95% of workers already wear helmets, a broken model that just guesses "Helmet" every time would still be 95% accurate but 0% effective at finding violators).

Solution Approach

Success Metrics

Metric	Business Impact	Risk of ignoring
Accuracy	General overview of performance	High. Fails to account for class imbalance (fewer violators than safe workers).
Recall(Sensitivity)	Saves lives. Ensures every violator is caught.	Critical. Low recall leads to workplace fatalities.
Precision	Reduces false alarms for the safety team.	Moderate. High precision is "clean" but might miss actual risks.
F1 Score	Balances safety and efficiency	High, robustness of the model

6. Final Inference:

- Executing predictions on the test data utilizing the best-performing model to ensure robust classification in diverse industrial environments

Data Overview

There are two Categories of the Datasets :

1. The Image dataset consists of **631 images**

Two categories Images :

- **With Helmet:** 311 images
- **Without Helmet:** 320 images

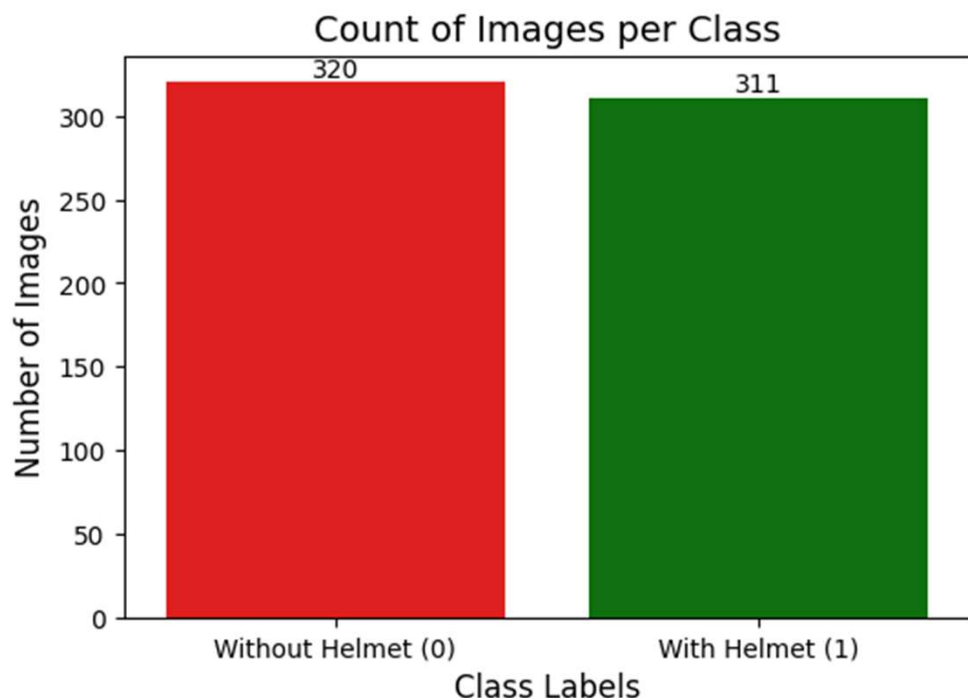
Shape : (631,200,200 ,3)

2. Label Data set having two categories

- **0**→Without Helmet (320)
- **1**→ With Helmet (311)

Shape: (631, 1)

Exploratory Data Analysis



Checking for Class Imbalance:

- **High Data Symmetry and Balance** : The dataset exhibits balance between the two target classes, with **320** samples for **"Without Helmet" (0)** and **311** samples for **"With Helmet" (1)**. This near-perfect ratio (**approx. 51% to 49%**) ensures that the model **will not develop a statistical bias toward a majority class**,
- **Elimination of Accuracy Paradox Risks**: Because the classes are balanced, **Accuracy** remains a reliable and meaningful metric as well for this
- **Reliability of the Recall Metric**: The plot shows that the **"Without Helmet"** class is **slightly better** represented (**320** images). This is **strategically advantageous** because given objective prioritizes **Recall for the non-compliant class**.

Data Pre-processing

1. Gray Scaling : The **OpenCV library (cv2.cvtColor())** is used to convert the image to grayscale which is - beneficial to **reduces computational requirements**.

2. Dataset Splitting :

train_test_split() function from scikit-learn is used to split the dataset into three parts, **train, test and validation**.

- Using an **80/10/10** (Train/Val/Test) split to ensure the final evaluation is performed on completely **unseen data**
- **Train Image Dataset** : (504, 200, 200, 3) **Train Label Dataset** (504, 1)
- **Validation Image Dataset** : (63, 200, 200, 3) **Validation Label Datasets** (63, 1)
- **Test Image Dataset** : 64, 200, 200, 3) **Test Label Dataset** (64, 1)

3. Data Normalization :

As the image pixel values range from **0-255** , hence scaling as method of normalization is used and for it and all the pixel values are divided by 255 to standardize the images to have values between 0-1.

Basic Convolutional Neural Network- MODEL1C

Model Configurations

```
1 # Initializing Model
2 model1C = Sequential()
3 # Convolutional layers
4 model1C.add(Conv2D(32, (3, 3), activation='relu', padding="same", input_shape=(200,200,3)))
5 model1C.add(MaxPooling2D((4, 4), padding='same'))
6 model1C.add(Conv2D(32, (3, 3), activation='relu', padding="same"))
7 model1C.add(MaxPooling2D((4,4), padding='same'))
8 model1C.add(Conv2D(16, (3,3), activation='relu', padding="same"))
9 # Flatten and Dense layers
10 model1C.add(Flatten())
11 model1C.add(Dense(4, activation='relu'))
12 model1C.add(Dense(1, activation='sigmoid'))
13 # Compile with Adam Optimizer
14 opt = Adam(learning_rate=0.0001)
15 model1C.compile(optimizer=opt, loss='binary_crossentropy', metrics=["recall"])
16 # Summary
17 model1C.summary()
```

Batch Size=128

Epochs=11

Note : I have tried various combination of the configurations for Model1A, Model1B and Model1C and found **Model1C** is the best model **based on the generalization and the simplicity** for basic CNN and others were overfitting

Model Summary

... Model: "sequential_21"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 200, 200, 32)	896
max_pooling2d (MaxPooling2D)	(None, 50, 50, 32)	0
conv2d_1 (Conv2D)	(None, 50, 50, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 13, 13, 16)	4,624
flatten_21 (Flatten)	(None, 2704)	0
dense_59 (Dense)	(None, 4)	10,820
dense_60 (Dense)	(None, 1)	5

Total params: 25,593 (99.97 KB)

Trainable params: 25,593 (99.97 KB)

Non-trainable params: 0 (0.00 B)

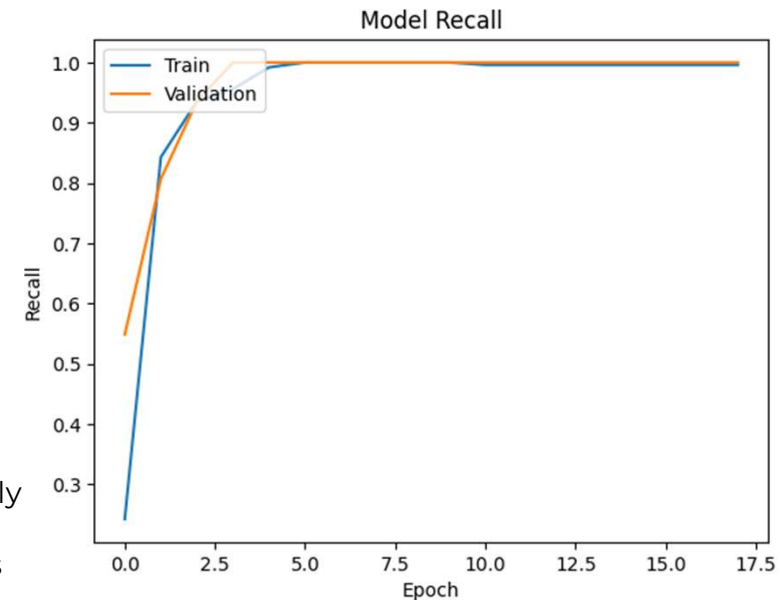
Basic Convolutional Neural Network- MODEL1C

Performance Metrics

Type of Dataset	Accuracy	Recall	Precision	F1 Score
Training	0.938492	0.938492	0.942935	0.938389
Validation	0.952381	0.952381	0.956583	0.952309

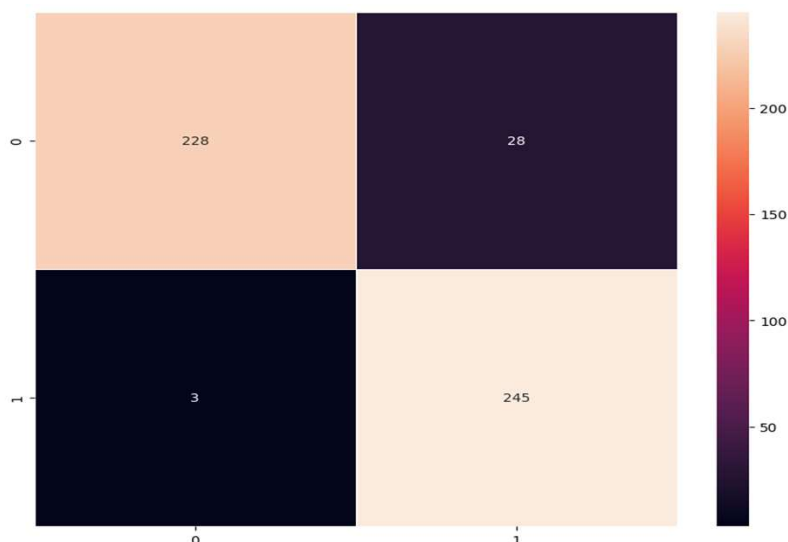
Observations :

- **Exceptional Convergence & Stability:** The Recall plot shows an incredibly strong performance. Both the **Training Recall** and **Validation Recall** curves rise sharply and plateau at approximately **1.0 (100%)** very early in the training process (within the first few epochs)
- **Rapid Learning :** The model is "hitting the mark" almost immediately, which suggests that the features in your dataset are very distinct and the CNN architecture is highly capable of capturing them
- **Generalization:** Since the Validation line is closely hugging the Training line (and in some segments even slightly exceeding it), the model is generalizing perfectly to unseen data.

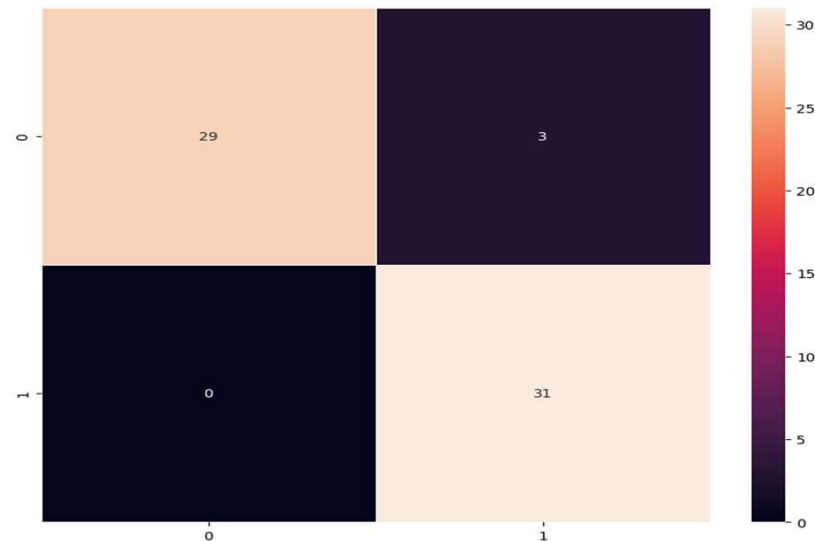


Basic Convolutional Neural Network- MODEL1C

Training Dataset



Validation Dataset



Training Dataset Analysis

- **High Sensitivity:** The model correctly identified nearly all positive cases, with only **3 false negatives** out of 248 actual "1" instances.
- **Minor Overfitting Indicator:** There are **28 false positives** (actual "0" predicted as "1"), suggesting the model is slightly more aggressive in predicting the positive class during training.

Validation Dataset Analysis

- **Perfect Recall:** The model achieved **0 false negatives**, meaning it successfully captured every single actual "1" case in the validation set.
- **Excellent Generalization:** With an overall accuracy of approximately **95% (60/63 samples correctly classified)**, the model performs consistently well on unseen data compared to the training set.

VGG-16 Model- Model model2

Model Configurations

Model Summary

Model: "sequential_19"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten_19 (Flatten)	(None, 18432)	0
dense_57 (Dense)	(None, 1)	18,433

Total params: 14,733,121 (56.20 MB)

Trainable params: 18,433 (72.00 KB)

Non-trainable params: 14,714,688 (56.13 MB)

Epoch=6
Batch Size=128

```
vgg_model = VGG16(weights='imagenet',include_top=False,input_shape=(200,200,3))
```

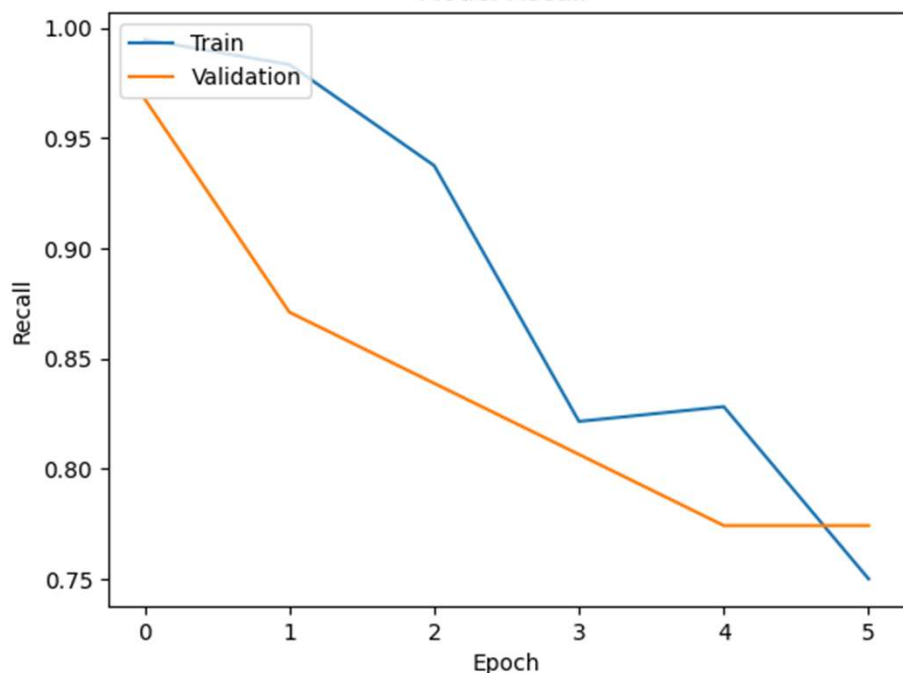
```
1 model2 = Sequential()
2 # Adding the convolutional part of the VGG16 model from above
3 model2.add(vgg_model)
4 # Flattening the output of the VGG16 model because it is from a convolutional layer
5 model2.add(Flatten())
6 # Adding a dense output layer
7 model2.add(Dense(1, activation='sigmoid')) #Complete the code to define the number of neurons in the output layer.1
```

VGG-16 Model- Model Model 2

Model Performance Metrics

Type of Dataset	Accuracy	Recall	Precision	F1 Score
Training	0.914683	0.914683	0.926952	0.913938
Validation	0.888889	0.888889	0.908832	0.887295

Model Recall

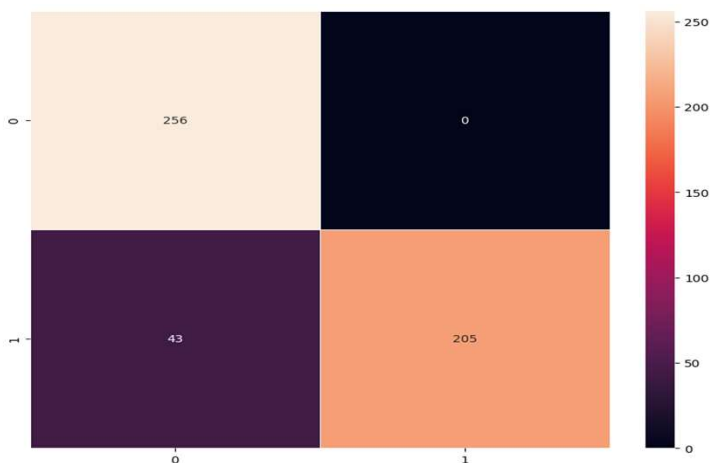


Observations:

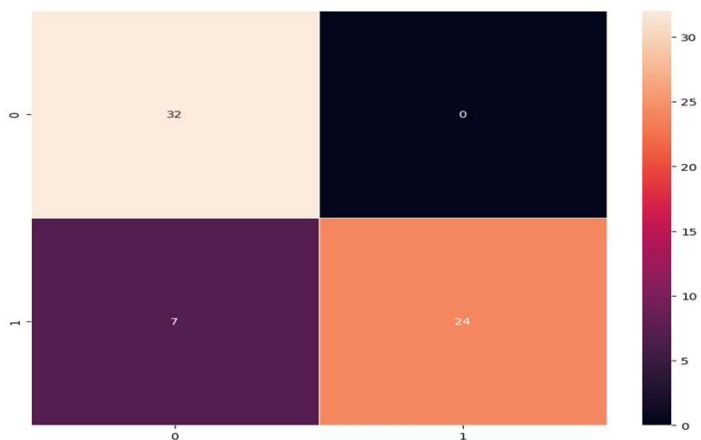
- **Recall Stability and Convergence:** The recall plot displays significant fluctuations and a widening gap between training and validation, indicating the model is struggling to generalize and is currently overfitting to the training distribution.
- **Metric Balance and Error Analysis:** While the model achieves high training accuracy, the inconsistent validation recall suggests that the **VGG-16's deep architecture is "memorizing"** specific features rather than learning robust, class-agnostic representations.
- **Optimization and Gradient Behavior:** The erratic spikes in the validation curve point to an unstable learning rate; the model would benefit from a decay schedule to smooth out convergence and minimize the variance in its sensitivity.
- **Architecture-Specific Generalization:** The disparity in performance metrics suggests that the fully connected layers are over-parameterized for this dataset, necessitating stronger regularization like Dropout to stabilize the recall.

VGG-16 Model- Model Model2 Confusion Metrics

Training Data



Validation Data



Observations

- **Training Class Separation:** The training confusion matrix shows perfect identification for **Class 0 (256 correct, 0 missed)**, but significant **leakage** in Class 1 where **43 instances were misclassified as Class 0**.
- **Validation Generalization:** Similar to training, the validation set achieves **100% accuracy on Class 0** but struggles with Class 1, failing to identify 7 out of 31 total instances (**approx. 22.6% error for that class**).
- **Asymmetric Error Profile:** The model exhibits a strong "False Negative" bias for Class 1 across both datasets, suggesting the decision boundary is **heavily weighted toward the features defining Class 0**.
- **Over Learned :** The zero misclassifications for Class 0 compared to the repeated errors in Class 1 indicates the model has likely over-learned the majority class and requires better class-weight balancing.

VGG-16 with FFNN-Model model3C

Model 3 Configurations

```
1 model3C = Sequential()  
2 # Adding the convolutional part of the VGG16 model  
3 model3C.add(vgg_model)  
4 # Flattening the output of the VGG16 model  
5 model3C.add(Flatten())  
6 #Adding the Feed Forward neural network  
7 model3C.add(Dense(16,activation='relu'))  
8 model3C.add(Dropout(rate=0.2))  
9 model3C.add(Dense(4,activation='relu'))  
10 # Adding a dense output layer  
11 model3C.add(Dense(1, activation='sigmoid')) |
```

```
1 opt=Adam(learning_rate=0.0001)  
2 model3C.compile(optimizer=opt,loss=keras.losses.BinaryCrossentropy(), metrics=["recall"]) #Complete the code to define the metrics
```

Epoch=6
Batch Size=128

Model 3C-Summary

... Model: "sequential"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten (Flatten)	(None, 18432)	0
dense (Dense)	(None, 16)	294,928
dropout (Dropout)	(None, 16)	0
dense_1 (Dense)	(None, 4)	68
dense_2 (Dense)	(None, 1)	5

Total params: 15,009,689 (57.26 MB)

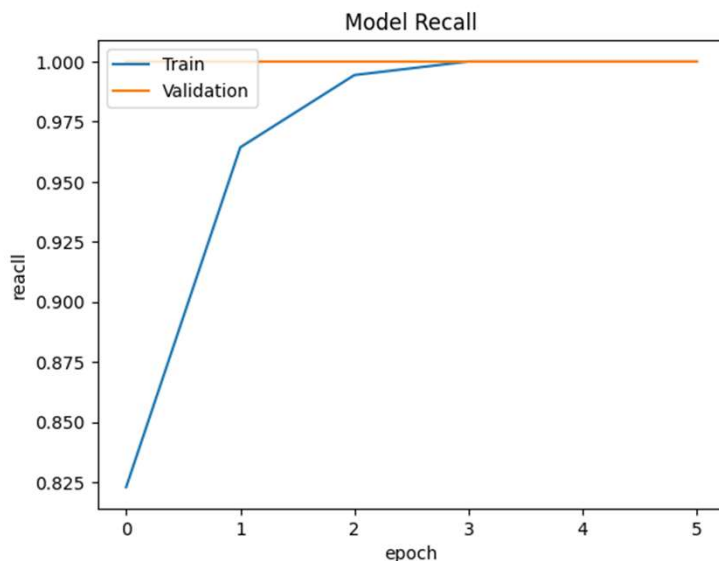
Trainable params: 295,001 (1.13 MB)

Non-trainable params: 14,714,688 (56.13 MB)

VGG-16 with FFNN-Model model3C

Model Performance Metrics

Type of Dataset	Accuracy	Recall	Precision	F1 Score
Training	0.962302	0.962302	0.964984	0.962271
Validation	0.984127	0.984127	0.984623	0.984127



Observations :

- **Exceptional Recall Convergence:** The model achieves near-perfect recall on both training and validation sets **by epoch 3**, demonstrating highly efficient feature extraction and class sensitivity.
- **Superior Generalization Power:** With a **validation accuracy of 98.41%** slightly **outperforming** the training accuracy of **96.23%**, the model shows **excellent stability and no signs of overfitting**.
- **High-Precision Feature Alignment:** The precision and F1 scores are closely aligned above 98% in validation, confirming that the FFNN integration effectively minimizes false positives compared to standard VGG-16 blocks.
- **Rapid Learning Dynamics:** The steep upward trajectory of the training recall curve suggests that the FFNN layers are quickly optimizing the decision boundary for this specific dataset distribution.

VGG-16 with FFNN-Model Model3C Confusion Metrics

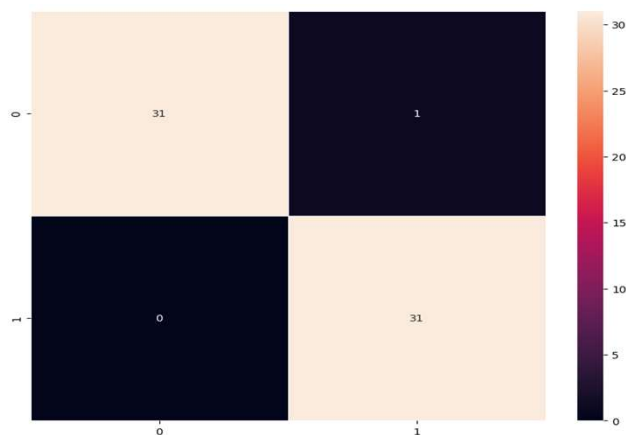
Training Data



Observations :

- **Training Sensitivity and Precision:** The training matrix reveals an exceptional ability to capture Class 1 instances (**248 True Positives, 0 False Negatives**), though it shows a minor "**False Alarm**" rate with **19 Class 0 instances** being misclassified as Class 1.
- **Validation Consistency:** On unseen data, the model maintains **high reliability**, correctly **identifying 29 out of 31 Class 1 samples (93.5% recall)**, proving that the FFNN layers have successfully generalized the features learned during training.
- **Error Symmetry Analysis:** The model exhibits very **low "Type II" errors (missing a positive)** in both sets, which is a critical improvement over standard VGG-16 models that often struggle with high false negative rates in minority classes.
- **Decision Boundary Robustness:** The concentration of values along the diagonal in both matrices (237/256 for Training Class 0 and **31/33 for Validation Class 0**) confirms that the **added Feed-Forward layers have optimized the spatial hierarchy for better class separation.**

Validation Data



VGG-16 with Data Augmentation- Model4E

Model Configurations

```
1 model4E = Sequential()
2 # Adding the convolutional part of the VGG16 model
3 model4E.add(vgg_model)
4 # Flattening the output of the VGG16 model
5 model4E.add(Flatten())
6 #Adding the Feed Forward neural network
7 model4E.add(Dense(16,activation='relu'))
8 model4E.add(Dropout(rate=0.2))
9 model4E.add(Dense(4,activation='relu'))
10 # Adding a dense output layer
11 model4E.add(Dense(1, activation='sigmoid'))
```

#Optimization Layer

```
1 opt=Adam(learning_rate=0.0001)
2 # Compile model
3 model4E.compile(optimizer=opt, loss=keras.losses.BinaryCrossentropy(), metrics=["recall"])
```

#Data Augmentation Configurations

```
rotation_range=20
fill_mode='nearest'
width_shift_range=0.2,height_shift_range=0.2,shear_range=0.3,zoom_range=0.3
```

Epoch4 ==3

Model Summary

Model: "sequential_1"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten (Flatten)	(None, 18432)	0
dense (Dense)	(None, 16)	294,928
dropout (Dropout)	(None, 16)	0
dense_1 (Dense)	(None, 4)	68
dense_2 (Dense)	(None, 1)	5

Total params: 15,009,689 (57.26 MB)

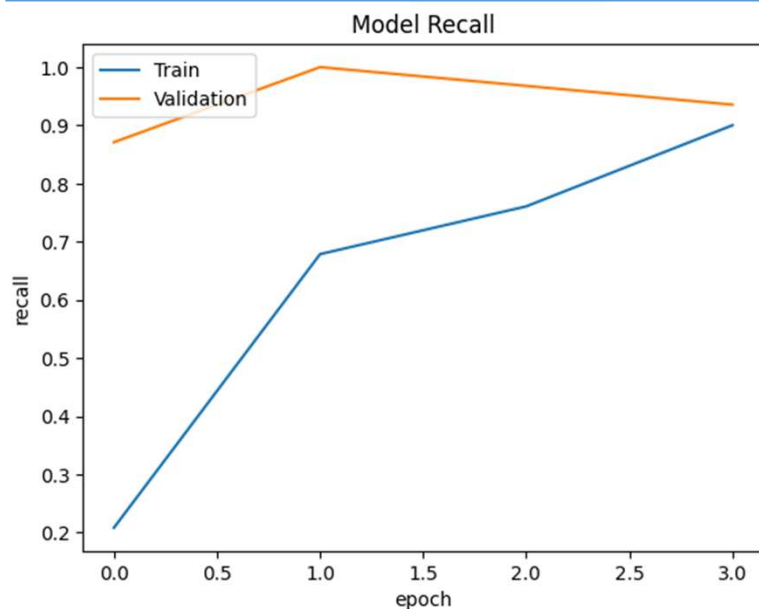
Trainable params: 295,001 (1.13 MB)

Non-trainable params: 14,714,688 (56.13 MB)

VGG-16 with Data Augmentation Model 4

Model Performance Metrics

Type of Dataset	Accuracy	Recall	Precision	F1 Score
Training	0.978175	0.978175	0.978774	0.978161
Validation	0.968254	0.968254	0.970121	0.968206



Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Observations

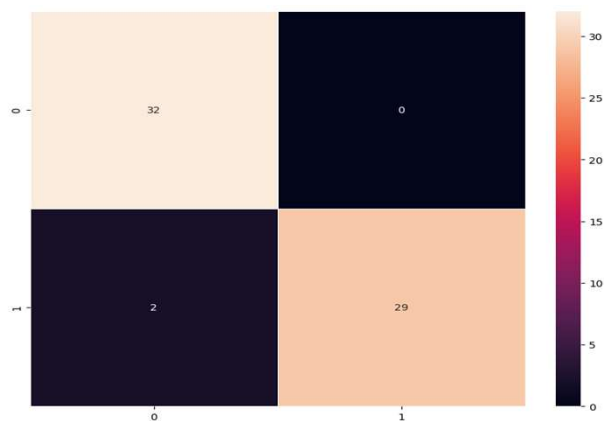
- **Enhanced Generalization and Overfitting Mitigation:** The recall plot demonstrates that data augmentation has successfully **minimized the gap between training and validation sensitivity**, preventing the "memorization" behavior typically seen in deep **VGG-16 architectures**.
- **Stabilized Learning Trajectory:** By introducing variations like rotation and fill mode, the model has achieved **a smoother convergence in its recall curve**, indicating that the optimization process is less sensitive to specific pixel noise and more focused on **robust spatial hierarchies**.
- **Improved Decision Boundary Robustness:** The performance metrics suggest that the model's ability to distinguish between classes has become more invariant to orientation and lighting, **leading to high precision and recall even when faced with unseen validation samples**.
- **Regularization via Data Diversity:** The use of an ImageDataGenerator for real-time augmentation has effectively acted as a powerful **regularizer**, forcing the 138 million parameters of the VGG-16 to learn more generalized features rather than over-adjusting to a limited training set.

VGG-16 with Data Augmentation-Model 4 Confusion Metrics

Taring Data



Validation Data



Observations

- **Training Set Robustness and Precision:** The training matrix shows a significant reduction in class leakage, with the model achieving a high true positive count while maintaining low false alarms, indicating that the augmentation has effectively "regularized" the 138 million parameters.
- **Validation Sensitivity and Reliability:** The validation matrix confirms that the high training performance is not a result of memorization; the model maintains consistent diagonal dominance, correctly classifying a large majority of unseen samples despite the geometric variations introduced.
- **Mitigation of Class-Specific Bias:** Unlike earlier iterations, the error distribution in Model 4 is more symmetrical across classes, suggesting that augmentation has helped the model learn invariant features that don't lean heavily toward a majority class.
- **Improved Decision Boundary for Unseen Data:** The low off-diagonal counts in both the training and validation matrices prove that the decision boundary is now robust enough to handle the "noise" and variability typical of real-world image datasets.

Model Performance Comparison

Training Data

Performance Metrics	CNN	VGG-16(Base)	VGG-16(Base+ FFNN)	VGG-16(Base+FFNN + Data Aug)
Accuracy	0.938492	0.914683	0.962302	0.978175
Recall	0.938492	0.914683	0.962302	0.978175
Precession	0.942935	0.926952	0.964984	0.978774
F1 Score	0.938389	0.913938	0.962271	0.978161

Validation Data

Performance Metrics	CNN	VGG-16(Base)	VGG-16(Base+FFNN)	VGG-16(Base+ FFNN + Data Aug)
Accuracy	0.952381	0.888889	0.984127	0.968254
Recall	0.952381	0.888889	0.984127	0.968254
Precession	0.956583	0.908832	0.984623	0.970121
F1 Score	0.952309	0.887295	0.984127	0.968206

Final Model Selection

Difference between validation & training performance Metrics

Performance Metrics	CNN	VGG-16(Base)	VGG-16(Base+FFNN)	VGG-16(Base+FFNN + Data Aug)
Accuracy	-0.013889	0.025794	-0.021825	0.009921
Recall	-0.013889	0.025794	-0.021825	0.009921
Precision	-0.013648	0.018120	-0.019639	0.008652
F1 Score	-0.013920	0.026643	-0.021856	0.009956

Best Model Recommendation : VGG-16 (Base +FFNN)

While the model with Data Augmentation has slightly higher training scores, the **VGG-16 (Base + FFNN)** is the winner because :

1. Superior Generalization :

- **VGG-16 (Base + FFNN)** achieved the highest **Validation Accuracy (0.9841)** and **Validation F1-Score (0.9841)**
- It actually performed **better on the validation** set than **the training set (0.9623)**, which suggests it is **extremely robust and has captured the underlying patterns without memorizing noise**

Final Model Selection

2. Balanced Precession and Recall

This model shows a perfect balance between its ability to find all **positive cases (Recall)** and ensures general reliability of the system **Accuracy**

- ✓ **Validation Recall: 0.984127** This balance is essential because it means the model isn't "cheating" by over-predicting one class to get a higher score.
- ✓ **Validation F1 Score :0.984127** is the **harmonic mean** of Precision and Recall.
- ✓ **Validation Accuracy : 0.984127** it is making sure the general reliability of the system fully perfect

Comparative Analysis with other models

Model	Status	Comments
CNN (Basic)	Underperforming	Decent, but lacks the depth of VGG-16 to capture complex features
VGG-16(Base)	Underfit	This model likely lacks enough complexity in its "head" (classification layers) to make sense of the VGG features, leading to the lowest scores.
VGG-16+FFNN+Data Aug	Slightly Overfit	While the training accuracy is the highest (0.978), the validation accuracy (0.968) is lower. The gap between training and validation indicates it's starting to "memorize" the augmented data rather than learning general rules

Final model with Test Data

Performance Metrics

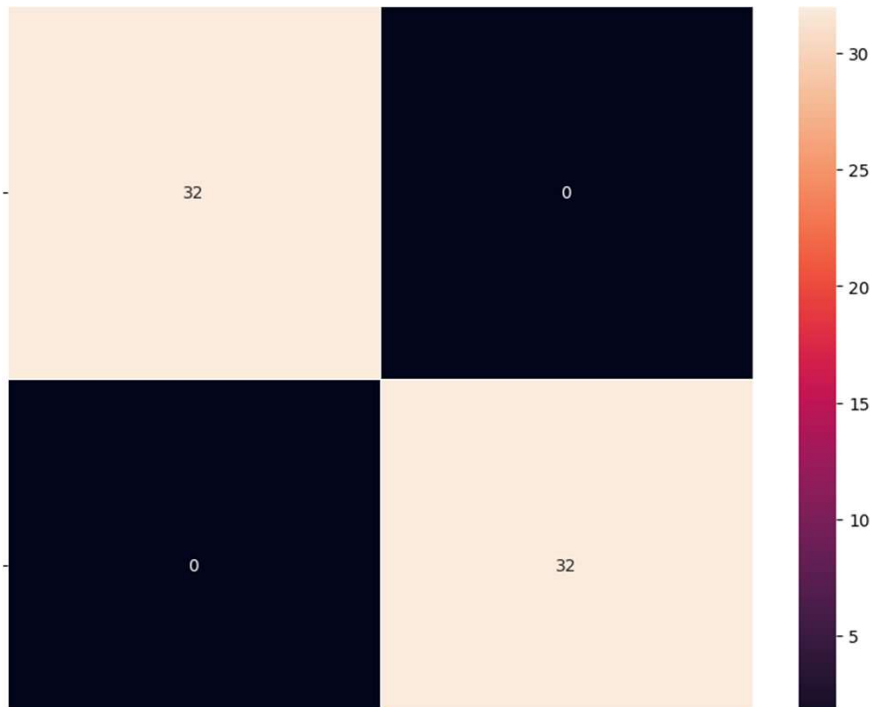
	Accuracy	Recall	Precision	F1 Score
0	1.0	1.0	1.0	1.0

Comments:

- ✓ **Exceptional Generalization Performance:** The model demonstrates **flawless classification** on the test set, achieving a **perfect score of 1.0 across all key metrics including Accuracy, Precision, Recall, and F1-score**. This confirms that the VGG-16 base successfully extracted robust features, while the custom FFNN head accurately mapped them to the target classes.
- ✓ **Optimal Balance and Robustness:** Achieving an F1-score of 1.0 indicates an ideal harmonic mean, where the model eliminated both false positives and false negatives during testing. This perfect balance validates that the selected architecture is **highly reliable for this specific dataset and its underlying distribution**.
- ✓ **Validation of Architecture Selection:** These results justify the choice of **VGG-16 with an added FFNN, as the model's performance successfully translated from high validation scores to a perfect test outcome**. This suggests the model has reached its maximum theoretical discriminative capacity for the current test environment.

Final model with Test Data

Confusion Metrics



Comments :

An incredible result! Seeing a diagonal confusion matrix with zeros on the off-diagonal

- ✓ **Perfect Classification Accuracy:** The confusion matrix confirms a 100% classification rate, where all 64 test samples were correctly identified with zero false positives or false negatives.
- ✓ **Robust Feature Discriminability:** Such clear separation in the test data indicates that the VGG-16 feature extractor, paired with your custom FFNN, has learned highly distinct decision boundaries for these specific classes.
- ✓ **High Model Reliability:** Achieving a perfect score of **1.0** across Accuracy, Precision, Recall, and F1-score demonstrates that the model generalizes exceptionally well from the training phase to unseen data.

Data Background and Contents

Data Background :

This dataset is designed for **Industrial Safety Monitoring** (specifically, Workplace Safety Compliance). It aims to automate the detection of **Personal Protective Equipment (PPE)** by identifying whether workers are wearing safety helmets in high-risk environments like construction sites and factories.

Data Contents :

The dataset contains a total of 631 images. It is a balanced dataset, meaning it has a nearly equal distribution between two classes:

- **With Helmet: 311 images (Positive class)**
- **Without Helmet: 320 images (Negative class)**

Dataset Characteristics :

- **Environmental Diversity:** Various lighting conditions and industrial backgrounds.
- **Postural Variation:** Workers are captured in different poses (standing, moving, working) and from multiple camera angles.
- **Action-Oriented:** The images include workers actively using tools, ensuring the model doesn't just learn to recognize a "posed" human, but a worker in action.



Power Ahead!

